PROJECT PLANNING & MANAGEMENT

AI-Powered Predictive Maintenance for Industrial Equipment

Dataset: IndFD-PM-DT

Industrial Fault Detection – Predictive Maintenance – Digital Twin

1. Project Proposal

1.1 Overview

This project aims to develop an AI-driven predictive maintenance system for industrial equipment using the IndFD-PM-DT (Industrial Fault Detection – Predictive Maintenance – Digital Twin) dataset.

The dataset is described as containing multi-sensor time-series readings, including vibration, temperature, pressure, acoustic signals, humidity, motor speed, torque, energy consumption, production rate, tool wear, and machine health indicators. It also includes diagnostic information such as failure mode indicators, fault probability, anomaly scores, and fault diagnosis labels.

The proposed system will use machine learning techniques to identify potential equipment faults and support early maintenance decisions. This can help reduce unexpected downtime, improve equipment reliability, and support more efficient maintenance planning.

1.2 Dataset Summary

Attribute	Details
Dataset Name	IndFD-PM-DT
File Name	IndFD-PM-DT dataset.csv
Number of Records	Approximately 1,000+ rows
Date Range	2019–2023

Sampling Frequency	Hourly, according to the project description
Total Columns	40
Target Column	Fault_Diagnosis
Target Classes	0, 1, 2, 3, 4
Supporting Labels	Failure_Mode_Indicators, Fault_Probability, Anomaly_Scores, Predictive_Maintenance_Scores

1.3 Key Dataset Features

A. Primary Sensor Features

- Vibration_Level
- Temperature_Readings
- Pressure_Data
- Acoustic_Signals
- Humidity_Levels

B. Operational Metrics

- Motor_Speed
- Torque_Data
- Energy_Consumption
- Production_Rate
- Tool_Wear_Rate
- Cycle_Time_Per_Operation
- Machine_Load_Percentage
- Machine_Utilization_Rate
- Idle_Time

C. Environmental Features

- Ambient_Temperature
- Humidity
- Air_Quality_Index

D. Machine Health and Diagnostic Features

- Machine_Health_Index
- Component_Degradation_Index
- Real_Time_Performance_Index
- Fault_Probability
- Anomaly_Scores

E. Events and Maintenance Logs

- Maintenance_Logs
- Repair_Logs
- Spare_Part_Usage
- Downtime_Incidents
- Fault_Trigger_Timestamps
- Alarm_Trigger_Data
- Machine_Start_Stop_Events
- Previous_Fault_Occurrences

1.4 Project Objectives

The main objectives of this project are to:

1. Develop an end-to-end predictive maintenance workflow using the IndFD-PM-DT dataset.
2. Analyze time-series sensor data to identify degradation patterns and abnormal machine behavior.
3. Perform feature engineering to extract useful information, including rolling statistics, rates of change, and degradation indicators.

4. Train and compare machine learning models, including Random Forest, XGBoost, and LSTM where appropriate.
5. Evaluate model performance using suitable classification metrics.
6. Develop a model deployment prototype using an API.
7. Implement experiment tracking, model monitoring, and a proposed retraining workflow.

1.5 Project Scope

In Scope

- Data collection and inspection.
- Data cleaning and preprocessing.
- Exploratory Data Analysis (EDA).
- Time-series analysis.
- Feature engineering.
- Multi-class classification using Fault_Diagnosis.
- Model training, tuning, and evaluation.
- Model deployment using FastAPI and Docker.
- Experiment tracking and data drift monitoring.

Out of Scope

- Physical installation of industrial sensors.
- Real-time industrial-scale data streaming.
- Full production-grade cybersecurity implementation.

1.6 Project Deliverables

No.	Deliverable	Responsible Team Member
1	Dataset Exploration Report	Malak Hany
2	EDA Notebook	Malak Hany

3	Cleaned and Preprocessed Dataset	Malak Hany
4	Advanced Analysis Report	Jana Mohamed
5	Visualizations and Dashboard	Jana Mohamed
6	Feature Engineering Summary and Feature-Ready Dataset	Jana Mohamed
7	Model Evaluation Report	Habiba Kamal
8	Python Model Training and Evaluation Code	Habiba Kamal
9	Final Model File and Reproducibility Notes	Habiba Kamal
10	Deployed API / Deployment Prototype	Sama Saed
11	MLOps and Deployment Report	Sama Saed
12	Monitoring and Retraining Setup	Sama Saed
13	Final Documentation and Project Demonstration	All Team Members

2. Project Plan and Timeline

2.1 Gantt Chart Overview

The project is planned to run from October 9, 2026, to December 4, 2026.

Phase	Task	Start Date	End Date	Duration
1	Project Planning and Management	9 Oct 2026	16 Oct 2026	1 week
2	Literature Review	9 Oct 2026	16 Oct 2026	1 week
3	Requirements Gathering	9 Oct 2026	16 Oct 2026	1 week
4	System Analysis and Design	9 Oct 2026	6 Nov 2026	4 weeks

5	Implementation (Source Code and Execution)	16 Oct 2026	30 Nov 2026	Approximately 6.5 weeks
6	Testing and Quality Assurance	16 Oct 2026	4 Dec 2026	Approximately 7 weeks
7	Final Presentation and Reports	16 Oct 2026	4 Dec 2026	Approximately 7 weeks

2.2 Weekly Milestones

Week	Milestone	Key Activities	Owner
Week 1	M1: Data Collection, EDA, and Preprocessing	Load the dataset, inspect columns, handle missing values and outliers, perform initial EDA, create basic rolling features, and save the cleaned dataset.	Malak Hany
Week 2	M2: Advanced Analysis and Feature Engineering	Analyze time-series trends and cycles, investigate correlated features, perform PCA where appropriate, engineer degradation features, and build visualizations.	Jana Mohamed
Week 3	M3: Model Development and Optimization	Define the target, perform chronological splitting, train and compare models, tune hyperparameters, and evaluate performance.	Habiba Kamal
Week 4	M4: MLOps, Deployment, and Monitoring	Track experiments, version the model, develop a FastAPI endpoint, package the application with Docker, and design monitoring and retraining workflows.	Sama Saed

2.3 Resource Allocation

Resource	Purpose
Python and Jupyter Notebook	Development and experimentation
IndFD-PM-DT.csv	Primary dataset
Pandas and NumPy	Data manipulation and numerical operations

SciPy and Scikit-learn	Statistical analysis and machine learning
Matplotlib and Seaborn	Data visualization
Plotly	Interactive visualizations
Random Forest and XGBoost	Classification models
LSTM	Potential sequence-based time-series model
MLflow	Experiment tracking and model management
DVC	Dataset and data pipeline versioning
FastAPI or Flask	API development
Docker	Application containerization
GitHub	Version control, collaboration, and documentation

3. Task Assignment and Team Roles

3.1 Team Responsibilities

1. Malak Hany — Data Engineer

Main responsibility: Data Collection, Exploration, and Preprocessing.

Tasks:

- Load the IndFD-PM-DT CSV file.
- Inspect the dataset structure and data types.
- Identify missing values and duplicate records.
- Investigate and handle outliers.
- Parse the Datetime column and sort the records chronologically.
- Perform initial Exploratory Data Analysis (EDA).
- Create basic features, such as rolling averages.
- Export the cleaned and preprocessed dataset.
- Prepare the Dataset Exploration Report and EDA Notebook.

2. Jana Mohamed — Data Analyst / Feature Engineer

Main responsibility: Advanced Analysis and Feature Engineering.

Tasks:

- Analyze time-series patterns in vibration, temperature, and pressure readings.
- Investigate trends and cyclical patterns.
- Explore correlations among sensor and operational variables.
- Apply Principal Component Analysis (PCA) when appropriate.
- Create rate-of-change features.
- Calculate rolling statistics, including mean, standard deviation, minimum, and maximum.
- Investigate component degradation patterns.
- Develop visualizations and an interactive dashboard.
- Prepare the Advanced Analysis Report and Feature Engineering Summary.

3. Habiba Kamal — Machine Learning Engineer

Main responsibility: Model Development, Training, and Evaluation.

Tasks:

- Define the prediction target using Fault_Diagnosis.
- Inspect target classes and verify label quality.
- Select appropriate input features and exclude unavailable-at-prediction-time information.
- Split the dataset chronologically into training, validation, and test sets.
- Train and compare Random Forest, XGBoost, and LSTM models where appropriate.
- Tune model hyperparameters.
- Evaluate model performance using Precision, Recall, F1-Score, and ROC-AUC where suitable.
- Analyze the confusion matrix and per-class performance.
- Select the final model based on validation results.
- Save the final model and prepare the Model Evaluation Report.

4. Sama Saed — MLOps Engineer

Main responsibility: Deployment, Monitoring, and Model Lifecycle Management.

Tasks:

- Implement experiment tracking using MLflow.
- Manage model versions and reproducibility.
- Develop a FastAPI or Flask prediction endpoint.
- Package the application using Docker.
- Implement input-data validation.
- Design a data drift monitoring workflow.
- Define model retraining triggers.
- Prepare the MLOps and Deployment Report.
- Support the final project demonstration.

3.2 Responsibility Matrix (RACI)

RACI defines the team's participation in each task.

- R — Responsible: Performs the task.
- A — Accountable: Ultimately accountable for completion.
- C — Consulted: Provides advice or technical input.
- I — Informed: Kept informed of progress.

Task	**Malak**	**Jana**	**Habiba**	**Sama**
Data Collection and Loading	R	C	I	I
EDA and Cleaning	R	C	I	I
Advanced Analysis and PCA	C	R	C	I
Feature Engineering	C	R	C	I
Model Development	I	C	R	C
MLOps and Deployment	I	I	C	R

Documentation	R	R	R	R

4. Risk Assessment and Mitigation Plan

No.	Risk	Likelihood	Impact	Mitigation Strategy
1	Missing values in sensor columns	High	Medium	Investigate missingness patterns and apply suitable imputation methods, such as median or time-aware forward filling.
2	Outliers in vibration and pressure readings	High	Medium	Investigate unusual readings and use appropriate outlier detection techniques. Avoid removing genuine fault signals automatically.
3	Class imbalance in Fault_Diagnosis	Medium	High	Use class weights and per-class evaluation. Consider resampling within the training data only.
4	Time leakage during dataset splitting	Medium	High	Use chronological splitting and ensure preprocessing and feature generation do not use future information.
5	Highly correlated features	Medium	Low	Inspect correlation matrices and consider feature selection, PCA, or VIF analysis when appropriate.
6	Model overfitting	Medium	Medium	Use validation data, regularization, appropriate cross-validation, and early stopping for LSTM where applicable.
7	API or deployment problems	Low	Medium	Test API endpoints early and validate the Docker deployment before the final demonstration.
8	Data or model drift after deployment	Medium	High	Monitor input distributions and prediction behavior; define alert thresholds and retraining criteria.

9	Dataset may be synthetic or simulated	To be verified	Medium	Verify the dataset's origin and document its limitations. Avoid treating simulated results as proof of industrial performance.
10	Team member unavailability	Low	Medium	Maintain shared documentation, version-controlled code, and knowledge sharing across the team.

5. Key Performance Indicators (KPIs)

The following KPIs define the project's proposed performance targets. They are goals to be evaluated, not guaranteed results.

KPI	Target	Measurement Method
Model Precision	≥ 85%	Test set
Model Recall	≥ 80%	Test set
F1-Score	≥ 0.82	Test set
ROC-AUC	≥ 0.90	Test set, using an appropriate multiclass evaluation method
MAE for Health Index Prediction	≤ 5.0	Validation set, if a separate health-index prediction task is implemented
API Response Time	< 500 ms	API performance and load testing
System Uptime	≥ 99%	Deployment monitoring, if continuously hosted
Data Drift Detection	Within 24 hours	Monitoring and alerting tests
Retraining Cycle	≤ 1 week	Time from receiving new labeled data to a validated model update
User Adoption Rate	≥ 70%	Stakeholder feedback, if applicable
Documentation Completeness	100%	Checklist-based GitHub repository review

6. Dataset-Specific Plan of Action

6.1 Week 1 — Data Engineering

Responsible: Malak Hany

Tasks

1. Load the IndFD-PM-DT.csv dataset.
2. Inspect the dataset dimensions, column names, and data types.
3. Parse the Datetime column and sort records chronologically.
4. Check missing values and duplicate records.
5. Investigate missing values in:
    - Ambient_Temperature
    - Humidity
    - Air_Quality_Index
6. Detect and investigate unusual readings in:
    - Vibration_Level
    - Pressure_Data
    - Acoustic_Signals
7. Perform initial EDA, including distributions and correlation analysis.
8. Create basic rolling mean features for vibration, temperature, and pressure.
9. Ensure that rolling features do not use future observations.
10. Save the cleaned dataset and EDA Notebook.

Deliverables

- Dataset Exploration Report.
- EDA Notebook.
- Cleaned and preprocessed dataset.

6.2 Week 2 — Advanced Analysis and Feature Engineering

Responsible: Jana Mohamed

Tasks

1. Create time-series plots for Vibration_Level, Temperature_Readings, and Pressure_Data.
2. Investigate trends and cyclical patterns.
3. Apply time-series decomposition, such as STL, if the data frequency and length support it.
4. Investigate correlations among Motor_Speed, Torque_Data, and Energy_Consumption.
5. Apply PCA to suitable scaled numerical features if it provides useful dimensionality reduction.
6. Engineer rate-of-change features, such as:
    - Change in vibration.
    - Change in temperature.
7. Analyze Component_Degradation_Index over time.
8. Calculate rolling statistics over 6-hour, 12-hour, and 24-hour windows, where appropriate.
9. Develop an interactive dashboard using Plotly and/or Streamlit.

Deliverables

- Advanced Analysis Report.
- Visualizations and dashboard.
- Feature Engineering Summary.
- Feature-ready dataset.

6.3 Week 3 — Machine Learning Model Development

Responsible: Habiba Kamal

Tasks

1. Define the prediction target

The primary target is Fault_Diagnosis, with five proposed classes: 0, 1, 2, 3, and 4.

The meaning of each class must be verified before interpreting model predictions.

2. Prepare the dataset

- Select suitable input features.
- Exclude target-derived columns and information that would not be available when a prediction is made.
- Split the data chronologically into training, validation, and test sets using a proposed 70/15/15 ratio.
- Fit preprocessing steps using training data only.

3. Train candidate models

- Random Forest Classifier.
- XGBoost Classifier.
- LSTM, if the dataset provides enough sequential observations for meaningful sequence learning.

4. Optimize model performance

- Use Grid Search or Randomized Search where appropriate.
- Use time-aware validation.
- Investigate class imbalance.
- Compare candidate models using validation results.

5. Evaluate the models

Calculate:

- Precision.
- Recall.
- F1-Score.
- Multiclass ROC-AUC, where appropriate.
- Confusion Matrix.
- Per-class performance.

6. Select and save the final model

Choose the model based on validation performance, generalization, and suitability for deployment.

Save the final model in .pkl format or another appropriate format, together with reproducibility notes.

Deliverables

- Model Training and Evaluation Code.
- Model Evaluation Report.
- Final Trained Model.
- Model Performance Comparison.

6.4 Week 4 — MLOps and Deployment

Responsible: Sama Saed

Tasks

1. Use MLflow to track experiments, parameters, metrics, and model artifacts.
2. Use DVC for dataset and pipeline versioning where appropriate.
3. Develop a FastAPI endpoint:

POST /predict

The endpoint will accept sensor data in JSON format and return the model's prediction.

4. Package the API using Docker.
5. Implement input validation and error handling.
6. Monitor data quality and investigate potential drift in variables such as:
    - Vibration_Level
    - Temperature_Readings
7. Define alert thresholds and a retraining workflow.
8. Test the complete prediction pipeline.
9. Prepare deployment documentation and a final demonstration.

Deliverables

- Deployed API or deployment prototype.

- MLOps and Deployment Report.
- Monitoring and Retraining Setup.
- Final Project Demonstration.

7. Expected Project Outcome

The expected outcome is an end-to-end machine learning prototype capable of analyzing industrial sensor data and classifying machine fault conditions.

The project will combine data engineering, exploratory analysis, feature engineering, machine learning, model evaluation, and deployment practices into a single workflow.

The final system is intended to demonstrate how predictive maintenance can support early fault identification and improve maintenance planning. Its actual effectiveness will depend on the quality and authenticity of the dataset, the validity of its labels, the prevention of data leakage, and the measured performance of the trained models.

8. Conclusion

This project provides a structured approach to developing an AI-powered predictive maintenance system for industrial equipment. By dividing responsibilities among data engineering, advanced analysis, machine learning, and MLOps, the team can develop and evaluate the system in manageable stages.

The project will conclude with a trained model, an evaluation report, a deployment prototype, monitoring documentation, and a final demonstration of the complete workflow.